Neural Networks 192 (2025) 107892

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

RoSwish: A novel Rotating Swish activation function with adaptive rotation around zero

Shengbo Zhang, Guoqiang Ren *

Swinburne College, Shandong University of Science and Technology, Jinan, 250031, China

ARTICLE INFO

Keywords:
Activation function
Fusion property
Neural network
Non-stationary signal
Stochastic process
Sine wave

ABSTRACT

This paper innovatively proposes the **Ro**tating **Swish** (RoSwish) activation function to address the limitations of existing activation functions in terms of nonlinear expressive capabilities, which is mathematically defined as $f(x) = (x + \alpha) \cdot sigmoid(\beta \cdot x) - 0.5 \cdot \alpha$. RoSwish integrates the advantageous features of Rectified Linear Unit (ReLU), Gaussian Error Linear Unit (GELU), Exponential Linear Unit (ELU), Parametric ReLU (PReLU), and Swish activation functions, achieving a unification of smoothness, gradient stability, self-gating effects, adaptability, etc. The core design is based on the Sigmoid gating mechanism, which introduces learnable parameters α and β to dynamically adjust the rotation degree of the Swish function, thereby obtaining more flexible and powerful nonlinear expressive capabilities.

Experimental results demonstrate that RoSwish outperforms existing activation functions across multiple application domains. In MNIST classification, it achieves at least a 16.02 % improvement in performance; in MNIST autoencoding, it reduces the MSE by at least 5 % under low learning rate; in tweet tagging, it improves performance by at least 3.65 % under medium to high learning rate, and when combined with batch normalization under low learning rate, it achieves a 30.19 % improvement; in ETTm2 time series prediction, it improves performance by at least 6.06 % under medium to low learning rate.

Additionally, inspired by non-stationary stochastic process theory, this study designs a series of activation functions based on non-stationary sine waves, aiming to enhance the neural network's ability to represent non-stationary signals. In the MNIST Classification task, this series of activation functions improves performance by at least 6.55 %.

Code is available in https://github.com/weirdoZHANG/RoSwish.

1. Introduction

Artificial neural networks, as mathematical models of biological neural networks (McCulloch & Pitts, 1943; Rosenblatt, 1958), have always been inspired by biology. The activation function, as a core component that embodies their important characteristics (Lederer, 2021), plays a decisive role in the learning capabilities of the network.

Early activation functions such as Sigmoid and Tanh suffered from the vanishing gradient problem, which limited the training effectiveness of deep networks. The introduction of the Rectified Linear Unit (ReLU) (Glorot et al., 2011) effectively alleviated this issue, and the phenomenon of neuron deactivation led to the proposal of Leaky ReLU (LReLU) (Maas et al., 2013) and Parametric ReLU (PReLU) (He et al., 2015). To enhance the noise resistance of neural networks in the deactivated state, researchers proposed improved solutions such as the Exponential Linear Unit (ELU) (Clevert, 2015) and the Scaled ELU (SELU) (Klambauer et al., 2017).

In recent years, inspired by stochastic regularization (Srivastava et al., 2014) and the gating mechanisms of Long Short-Term Memory (LSTM) (Hochreiter, 1997), novel activation functions such as the Gaussian Error Linear Unit (GELU) (Hendrycks & Gimpel, 2016), Sigmoid-Weighted Linear Unit (SiLU) (Elfwing et al., 2018), and Mish (Misra, 2019) have been developed to dynamically adjust the features of input information, further improving the generalization capabilities of networks.

Research has found that trainable activation functions are less prone to overfitting and exhibit faster convergence and higher accuracy (Apicella et al., 2021; Godfrey, 2019). For example, the Adaptive Piecewise Linear Unit (APLU) (Agostinelli, 2014), PReLU (He et al., 2015), Parametric ELU (PELU) (Trottier et al., 2017), and Swish (Ramachandran et al., 2017) can adaptively learn the shape of the function based on

* Corresponding author.
E-mail address: renguoqiang@sdust.edu.cn (G. Ren).

https://doi.org/10.1016/j.neunet.2025.107892
Received 10 December 2024; Received in revised form 26 March 2025; Accepted 19 July 2025
Available online 26 July 2025
0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

data characteristics, thereby flexibly adapting to different datasets and learning tasks.

Building on the above research, this paper proposes a new comprehensive activation function, the Rotating Swish (RoSwish). This function significantly enhances the nonlinear expressive capabilities of networks through an adaptive rotation mechanism. The main contributions of this study are as follows:

- Proposing the RoSwish activation function, which achieves effective fusion and adaptive adjustment of multiple advantageous properties.
- Validating the significant performance advantages of RoSwish in computer vision, natural language processing, and time series prediction tasks.
- Designing a series of activation functions based on non-stationary sine waves (NsSin), providing new insights for non-stationary signal processing (See Appendix A).

The paper is organized as follows. Section 2 briefly reviews traditional nonlinear activation functions. Section 3 presents the proposed RoSwish activation function. Section 4 describes the parameter optimization method for RoSwish. Section 5 verifies the universal function approximation capability and the parameterized information gain of the RoSwish network. Section 6 shows the performance evaluation experiments and feature ablation experiments of RoSwish. Section 7 summarizes the experimental conclusions and discussions. Section 8 concludes the paper and outlines future prospects.

2. Related work

This section systematically reviews the development of activation functions.

The earliest linear activation functions (AFs) achieved linear mapping through the product of a constant and the input (Dubey et al., 2022), but failed to introduce the necessary nonlinearity, limiting the performance of deep networks.

The nonlinear Sigmoid function was widely adopted for its ability to simulate the firing characteristics of biological neurons (Dayan & Abbott, 2005), while Tanh is a scaled and shifted version of Sigmoid. Sigmoid compresses the input to the range [0, 1], but its non-zero mean property may restrict the efficiency of weight updates. In contrast, Tanh achieves a zero-mean output, effectively alleviating convergence issues.

Neuroscientific research has shown that neurons exhibit sparsity when encoding information, which has led to the widespread use of the ReLU function (Glorot et al., 2011). ReLU performs an identity mapping for positive values and outputs zero for negative values, causing negative neurons to remain zero during training. To improve training efficiency, LReLU (Maas et al., 2013) introduced a small non-zero slope in the negative region.

In noisy environments, AFs capable of handling negative values help reduce information loss and enhance network robustness. Although LReLU allows negative activations, it does not ensure noise-robust deactivation states, and experiments show its performance is comparable to ReLU (Maas et al., 2013). On the other hand, ELU (Clevert, 2015) saturates to negative values for smaller inputs, bringing the mean activation closer to zero and thereby improving training efficiency. SELU (Klambauer et al., 2017) further scales ELU to induce self-normalization in activation layers, significantly enhancing network robustness.

Since the output of Sigmoid can be interpreted as a probability, several probability-based AFs have been proposed. SiLU (Elfwing et al., 2018) achieves probabilistic activation by weighting the input. GELU (Hendrycks & Gimpel, 2016) uses the cumulative distribution function of the standard normal distribution as a gating mechanism, viewed as a modified expectation of adaptive dropout (Ba & Frey, 2013). Mish (Misra, 2019) combines Tanh with Softplus (Dugas et al., 2000), exhibiting self-regularization and non-monotonic properties similar to SiLU and GELU. Interestingly, ReLU can also be seen as a simple hard-gating mechanism based on probability, while SiLU, GELU, and Mish resemble continuous and "undershooting" versions of ReLU (Hahnloser et al., 2000).

Parametric AFs dynamically adjust trainable parameters, leveraging the uniqueness and complexity of various data to further enhance network performance. For example, APLU (Agostinelli, 2014) uses only a subset of the parameters of Maxout network (Goodfellow et al., 2013) and is trained using standard gradient descent (Fearnley et al., 2022), achieving significant improvements in accuracy (Godfrey, 2019). PReLU, as a parametric version of LReLU, employs momentum optimization to train parameters and outperforms ReLU and LReLU on the ImageNet task (He et al., 2015; Russakovsky et al., 2015). Similarly, PELU also uses standard gradient descent, with experimental results showing its superiority over ELU on CIFAR-10, CIFAR-100, and ImageNet tasks (Trottier et al., 2017). Additionally, Swish can be considered a parametric version of SiLU, often demonstrating higher accuracy across various datasets, learning tasks, and neural network models (Ramachandran et al., 2017). More importantly, parametric AFs not only improve model performance but also effectively mitigate the vanishing gradient problem (Hu et al., 2019).

Although existing AFs have partially addressed challenges such as nonlinear mapping, sparse representation, gradient stability, noise robustness, and adaptability, they still exhibit certain limitations. To address this issue, the proposed RoSwish cleverly integrates numerous advantages, including smoothness, non-monotonicity, non-saturation, gradient stability, noise robustness, self-gating effects, self-regularization, and adaptability. Unlike traditional parametric AFs, RoSwish's parameters are trained using the Adam optimizer (Kingma, 2014). Experimental results demonstrate its significant performance improvements across multiple datasets and tasks, providing a new direction for future network optimization.

3. RoSwish

This section introduces the proposed RoSwish activation function and its similarities to commonly used AFs. RoSwish, as a comprehensive activation function, is mathematically defined as:

$$f(x) = (x + \alpha) \cdot sigmoid(\beta \cdot x) - 0.5 \cdot \alpha \tag{1}$$

where $sigmoid(x) = (1 + e^{-x})^{-1}$. α and β represent channel-wise trainable parameters, jointly regulating the positive slope, negative saturation value, and negative concavity. The function remains continuously differentiable for any real-valued combination of α and β, which facilitates gradient flow through the network and reduces information loss. Fig. 1 illustrates how these parameters shape the RoSwish curve. Through in-depth research, it has been discovered that when α and β take specific values, RoSwish can be simplified or approximated to well-known existing AFs. For example, when $\alpha = 1$ and $\beta = 0$, RoSwish simplifies to SiLU or Swish-1. When α and β take other specific combinations, RoSwish can approximate the micro-rotated form of Mish near the origin or maintain small absolute differences with GELU, ELU, and SELU.

As shown in Fig. 3 (Left), the obvious negative concavity between RoSwish and some AFs is further demonstrated, a feature that helps to retain the information of small negative weights (Misra, 2019), which fundamentally benefits from the gating effect of Sigmoid or Tanh. As shown in Fig. 3 (Right), RoSwish and some AFs exhibit derivative saturation towards 0 or 1 as the input values approach the extremes. This characteristic effectively mitigates the vanishing and exploding gradient problems in deep network training.

Due to the algebraic relationship $tanh(x) = 2 \cdot sigmoid(2 \cdot x) - 1$, the definition of RoSwish can also be expressed as:

$$f(x) = x \cdot sigmoid(\beta \cdot x) + 0.5 \cdot \alpha \cdot tanh(0.5 \cdot \beta \cdot x) \tag{2}$$

From Equation (2), it can be seen that RoSwish not only incorporates the sigmoid gating mechanism but also allows the neural network to transmit information in the negative region. Similar to LReLU and

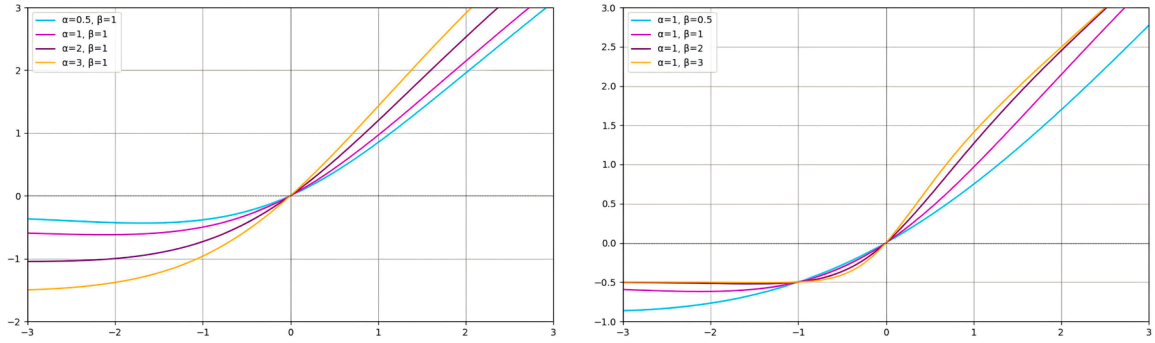


Fig. 1. RoSwish activation function under variations in α (Left) and β (Right).

ELU, RoSwish removes the hard-zero sparsity (Glorot et al., 2011) to potentially achieve more robust gradient properties. This trade-off enables RoSwish to better handle noise and anomalies during training, thereby improving the generalization capabilities of the neural network. However, compared to LReLU, RoSwish ensures a noise-robust deactivation state. Compared to ELU, RoSwish is smoother in the vicinity of the zero point.

4. Parameter optimization

This section introduces the parameter optimization method for RoSwish, as well as parameter initialization.

The learnable parameters α and β of RoSwish serve a dual regulatory role: they control the shape of the function and reduce the uncertainty of the output due to data features (See Sections 5.2 and 6.2.4). During optimization, these two parameters can be updated simultaneously with other parameters in the neural network through the backpropagation algorithm (Rumelhart et al., 1986). The parameter gradients for a layer can be derived using the chain rule as follows:

$$\frac{\partial \mathcal{E}}{\partial \alpha} = \frac{\partial \mathcal{E}}{\partial f(x)} \frac{\partial f(x)}{\partial \alpha} \tag{3}$$

$$\frac{\partial \mathcal{E}}{\partial \beta} = \frac{\partial \mathcal{E}}{\partial f(x)} \frac{\partial f(x)}{\partial \beta} \tag{4}$$

where $\mathcal{E}$ represents the objective function, and $\frac{\partial \mathcal{E}}{\partial f(x)}$ denotes the gradient propagated back from deeper layers of the network. Furthermore, the gradients of RoSwish with respect to parameters α and β are expressed as:

$$\frac{\partial f(x)}{\partial \alpha} = sigmoid(\beta \cdot x) - 0.5 \tag{5}$$

$$\frac{\partial f(x)}{\partial \beta} = x \cdot (x + \alpha) \cdot sigmoid(\beta \cdot x) \cdot (1 - sigmoid(\beta \cdot x)) \tag{6}$$

In the experiments conducted in this paper, the initial values for parameters α and β are set to 0.817 and 3, respectively. During training, no constraints are imposed on the range of parameter variations, allowing RoSwish to explore optimal parameter combinations within a broad range.

5. Theoretical analysis

5.1. Universal approximation theory

The Universal Approximation Theorem states that a feedforward neural network with at least one hidden layer can serve as a universal function approximator, provided that a class of AFs satisfying certain conditions is chosen (Augustine, 2024). To facilitate a clear proof that the proposed RoSwish function belongs to this class of AFs, this section first defines the following three theorems before presenting the proof.

Theorem 1 (Taylor's Formula Mathematics (2001), Taylor (1715))**.** *Any continuous function $f(x) : \mathbb{R} \to \mathbb{R}$ that is k-times differentiable at $a \in \mathbb{R}$ can be expressed as a sum of polynomials:*

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2}(x-a)^2 + \cdots + \frac{f^k(a)}{k!}(x-a)^k + R_k(x)$$
$$= \sum_{i=0}^{k} c_i (x-a)^i + R_k(x) \tag{7}$$

where $c_i = \frac{f^i(a)}{i!} = \frac{1}{i!}\frac{d^i}{dx^i} f(x)|_{x=a}$ and $R_k(x) = o(|x-a|^k)$ is the residual term. When $a = 0$, a special case of Equation (7) is:

$$f(x) = \sum_{i=0}^{k} c_i x^i + R_k(x) \tag{8}$$

this is the more commonly used version, known as the Maclaurin series.

Theorem 2 (Density of Non-Polynomial Functions (Leshno et al., 1993))**.** *Let $\mathbb{X}$ be any compact subset of $\mathbb{R}^n$, and ϕ is an activation function, then if and only if ϕ is not a polynomial function, the finite sum:*

$$f_\phi(x) = A_2 \phi(A_1 x + b_1) = \sum_{j=1}^{m_1} a_{2_j} \phi(A_{1_j} x + b_{1_j}) \tag{9}$$

is dense in $\mathbb{X}$.

Theorem 3 (Kolmogorov-Arnold Representation Theorem (Kolmogorov, 1957))**.** *Any continuous multivariate function $f : [0,1]^n \to \mathbb{R}$ can be expressed as a combination of a finite number of one-dimensional functions:*

$$f(x) = f(x_1, x_2, \ldots, x_n) = \sum_{j=1}^{2n+1} \beta_j \left(\sum_{i=1}^{n} \alpha_{ij}(x_i) \right) \tag{10}$$

where continuous functions $\alpha_{ij} : [0,1] \to \mathbb{R}$ for $i = 1, 2, \ldots, n$ and $j = 1, 2, \ldots, 2n+1$, $\beta_j : \mathbb{R} \to \mathbb{R}$ is a continuous function for each $j \in \{1, 2, \ldots, 2n+1\}$.

Now, the theory of RoSwish networks as universal function approximators can be established using the above three concepts. The theory is defined as follows:

Theorem 4 (Universal Approximation Capability of RoSwish Networks)**.** *Given the RoSwish function: $RoSwish(x) = (x + \alpha) \cdot sigmoid(\beta \cdot x) - 0.5 \cdot \alpha$, where $\alpha \in \mathbb{R}$ and $\beta \in \mathbb{R}$ are trainable parameters initialized as $\alpha_d = 0.817(\forall d)$ and $\beta_d = 3(\forall d)$, and d is the layer dimension, then for any $f : \mathbb{X} \to \mathbb{R}$ and $\epsilon > 0$, there exists a finite sum $f_{RoSwish}(x)$ as shown in Equation (9) such that for all $x \in \mathbb{X}$:*

$$|f(x) - f_{RoSwish}(x)| < \epsilon \tag{11}$$

In other words, a single-layer neural network based on the RoSwish function can approximate any continuous univariate function on a bounded domain with arbitrary precision, and further extend to multidimensional scenarios by combining with Theorem 3. Similarly, this holds for multilayer RoSwish networks.

Proof. Verifying the non-polynomial properties of RoSwish is the first step, and RoSwish consists of a linear term $(x+\alpha)$ multiplied by the Sigmoid function, subtracting the constant term 0.5α. Therefore, the problem reduces to proving the non-polynomial nature of the Sigmoid function. $Sigmoid(\beta x) = \frac{1}{1+e^{-\beta x}}$ is an analytic function, and its Taylor expansion is:

$$Sigmoid(\beta x) = \frac{1}{2} + \frac{\beta x}{4} - \frac{\beta^3 x^3}{48} + \cdots \quad (\beta \neq 0) \tag{12}$$

contains an infinite number of terms and cannot be simplified into a finite polynomial, meaning that when $\beta_d \neq 0$, $Sigmoid(\beta_d x)$ is a non-polynomial function.

Thus, when $\beta_d \neq 0$, RoSwish is a non-polynomial function, satisfying the core condition of Theorem 2. Applying Equation (9), we obtain:

$$f_{RoSwish}(x) = \sum_{j=1}^{m_1} a_{2_j} RoSwish(A_{1_j} x + b_{1_j}) \tag{13}$$

which can approximate any one-dimensional continuous function with arbitrary precision, satisfying Equation (11).

In this step, if $\beta_d = 0(\forall d)$, then $Sigmoid(0) = 0.5$, and RoSwish degenerates into a linear function $0.5x$, causing it to become a first-order polynomial and lose its universal approximation capability. To address this, the following Lemma 1 must be added to assist the proof.

Lemma 1. *When $\beta \in \mathbb{R}^d$ is a trainable parameter and satisfies the following condition, the RoSwish network can still achieve universal approximation: there exists at least one nonlinear neuron $(\beta_d \neq 0)$ in the network.*

Proof. Theorem 2 requires that the AF itself is a non-polynomial function. If the AF degenerates into a linear function in a certain dimension, it does not satisfy the theorem's condition in that dimension. However, the theorem only requires that the AF as a whole is non-polynomial, not that every neuron is non-polynomial. Therefore, as long as there exists at least one $\beta_d \neq 0$ where RoSwish remains a non-polynomial function, the network is still dense, and its overall universal approximation capability can be achieved through the combination of nonlinear parts. In practice, self-regularization or gradient optimizers can prevent β_d from being optimized to 0, making it almost impossible for the network to degenerate into a purely linear model. Since the expressive power of a linear model is far below the requirements of the target task, the optimization process naturally retains some or all $\beta_d \neq 0$ to maximize the network's expressive power. This conjecture is validated in Section 6.2.7. □

Based on the rigorous theoretical analysis of Lemma 1, the inherent limitations observed in RoSwish networks are systematically eliminated during the construction of Theorem 2, thereby strictly guaranteeing the universal approximation capability of RoSwish networks for any one-dimensional function. Next, the extension to multidimensional scenarios is achieved:

Promotion 1. *It is important to note that the target function $f : \mathbb{X}^n \to \mathbb{R}$ can be appropriately rescaled to facilitate the application of Theorem 3. For each α_{ij} and β_j, the one-dimensional approximation capability of RoSwish is utilized to construct a single-layer network ψ_{ij} and θ_j, such that:*

$$|\alpha_{ij}(x_i) - \psi_{ij}(x_i)| < \epsilon \tag{14}$$

$$|\beta_j(t_j) - \theta_j(t_j)| < \epsilon \tag{15}$$

Thus, Theorem 3 is used to combine a multidimensional approximation network, defined as:

$$f_{RoSwish}(x) = f(x_1, x_2, \ldots, x_n) = \sum_{j=1}^{2n+1} \theta_j \left(\sum_{i=1}^{n} \psi_{ij}(x_i) \right) \tag{16}$$

At the same time, the approximation error is also combined from one-dimensional to multidimensional, such that for any $f : \mathbb{X}^n \to \mathbb{R}$ and $\delta > 0$, for all $x \in \mathbb{X}^n$:

$$|f(x) - f_{RoSwish}(x)| < \delta \tag{17}$$

which concludes that RoSwish based single-layer network is a universal function approximator.

In fact, this also holds for multilayer networks (Augustine, 2024). This means that the proof of Theorem 4 is complete. □

5.2. Quantization of information gain

By introducing learnable parameters, the adaptive activation function can significantly improve the amount of information in the information theory framework. The following two definitions need to be given before the proof in this paper.

Definition 1. The feature transformation of a certain activation layer in the network is defined as:

$$Y = f(X; \theta) \tag{18}$$

where $X \in \mathcal{X}$ is the input random variable, following distribution $p(x)$, $\theta \in \Theta$ is the adaptive parameter, following distribution $q(\theta)$ induced by the optimization process, and $Y \in \mathcal{Y}$ is the output random variable, obtained through AF mapping.

Definition 2 (Information Gain for Parametric AFs)**.** From the perspective of information theory, information gain can be quantitatively represented as:

$$I(X;Y) = H(Y) - H(Y|X) \tag{19}$$

Parameterized expansion to:

$$\begin{aligned} I(X;Y;\theta) &= I(X;Y) + I(Y;\theta|X) = [H(Y) - H(Y|X)] + [H(Y|X) \\ &\quad - H(Y|X,\theta)] = H(Y) - H(Y|X,\theta) \end{aligned} \tag{20}$$

When f is a fixed AF, θ is a fixed constant, $H(Y|X,\theta) = H(Y|X)$, thus $I(X;Y;\theta) = I(X;Y)$. When f is an adaptive AF, θ is an optimizable variable, and the conditional entropy decreases: $H(Y|X,\theta) \leq H(Y|X)$. Therefore, the information gain of parametric AFs can be expressed as:

$$\Delta I = I(X;Y;\theta) - I(X;Y) = H(Y|X) - H(Y|X,\theta) \geq 0 \tag{21}$$

Next, we will roughly derive the lower bound of information gain for adaptive AFs. As shown in Fig. 2, each parameter space approximately follows a continuous Gaussian distribution. In the following derivation, parameters are still represented by θ and assumed to be independent of input X, expressed as:

$$\theta \sim \mathcal{N}(\mu, \Sigma), \quad p(x,\theta) = p(x)q(\theta) \tag{22}$$

where $q(\theta) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp(-\frac{1}{2}(\theta-\mu)^T \Sigma^{-1}(\theta-\mu))$ is the Gaussian probability density function, and d is the dimension of the parameter space.

Thus, the information gain is further expanded as:

$$\Delta I = \iiint p(x,\theta)p(y|x,\theta) \log p(y|x,\theta)\, dx\, dy\, d\theta - \iint p(x,y) \log p(y|x)\, dx\, dy \tag{23}$$

By extending the relationship between differential entropy and KL divergence, the conditional entropy difference is expanded as:

$$\Delta I = \mathbb{E}_{\theta \sim q(\theta)}[D_{KL}(p(y|x)||p(y|x,\theta))] \tag{24}$$

If $p(y|x,\theta)$ is assumed to be a Gaussian distribution, its differential entropy is:

$$h(Y|X,\theta) = \frac{1}{2} \log((2\pi e)^k |\Sigma_y|) \tag{25}$$

where k is the dimension of Y, and Σ_y is the conditional covariance matrix.

Finally, combining the parameter optimization error upper bound ϵ (assuming $\mathbb{E}[D_{KL}] \leq \epsilon$), we obtain:

$$\Delta I \geq -\frac{1}{2} \log |\Sigma| - C \cdot \epsilon \tag{26}$$

where C is a constant, and $|\Sigma|$ is the determinant of the parameter covariance matrix.

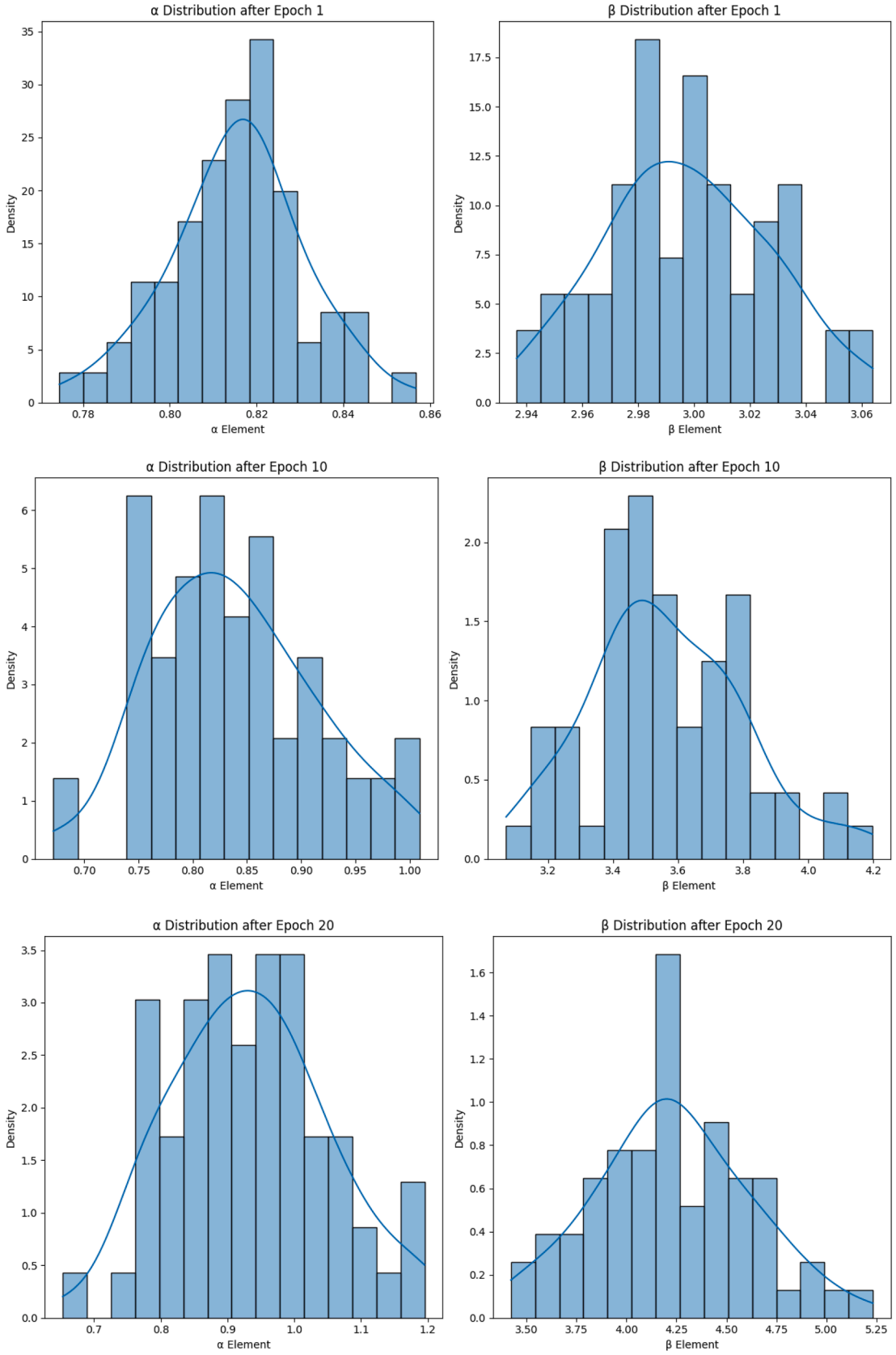


Fig. 2. Distribution of the parameter space of RoSwish during the training process on the ETTm2 dataset.

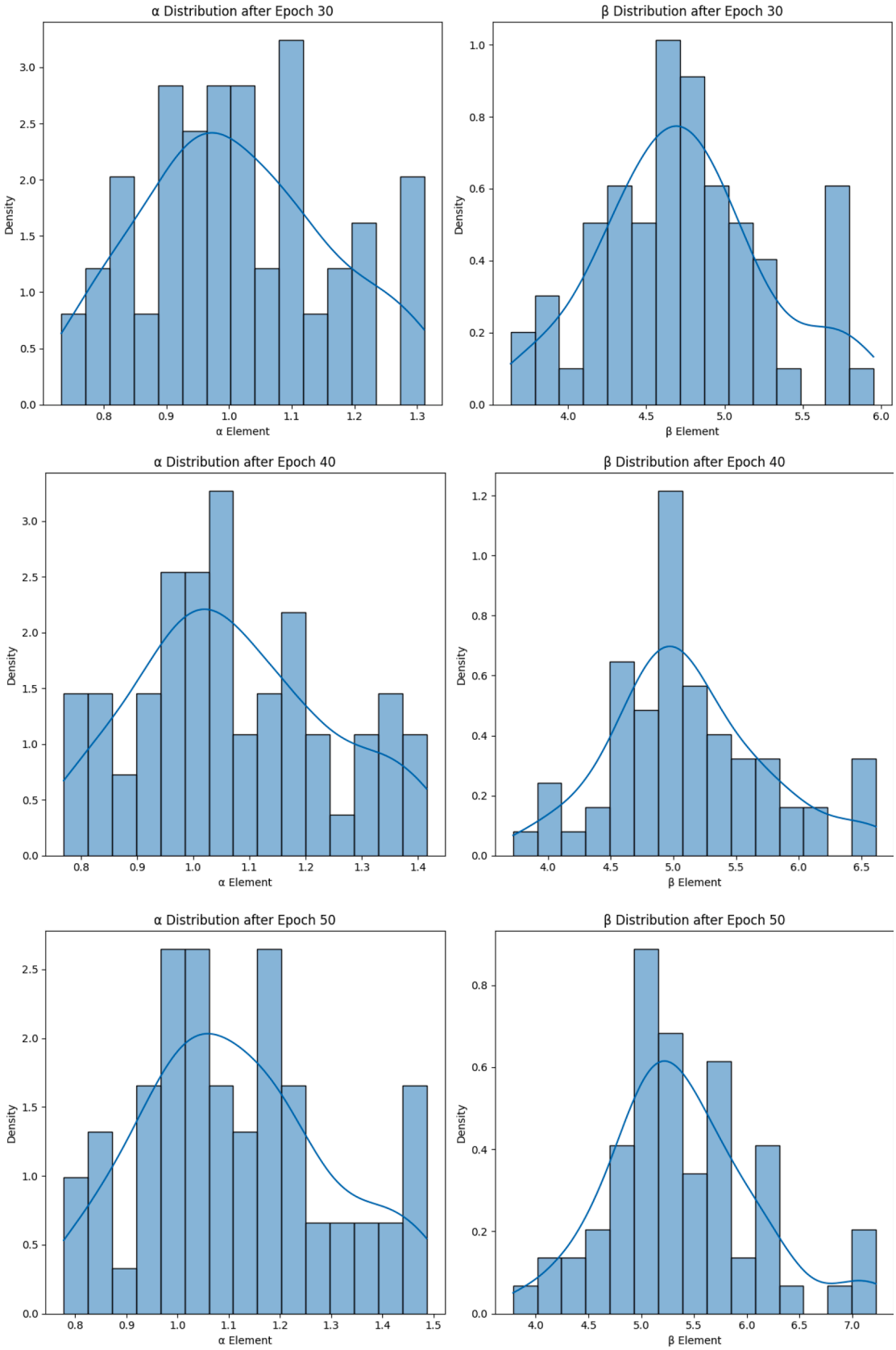


Fig. 2. Continued.

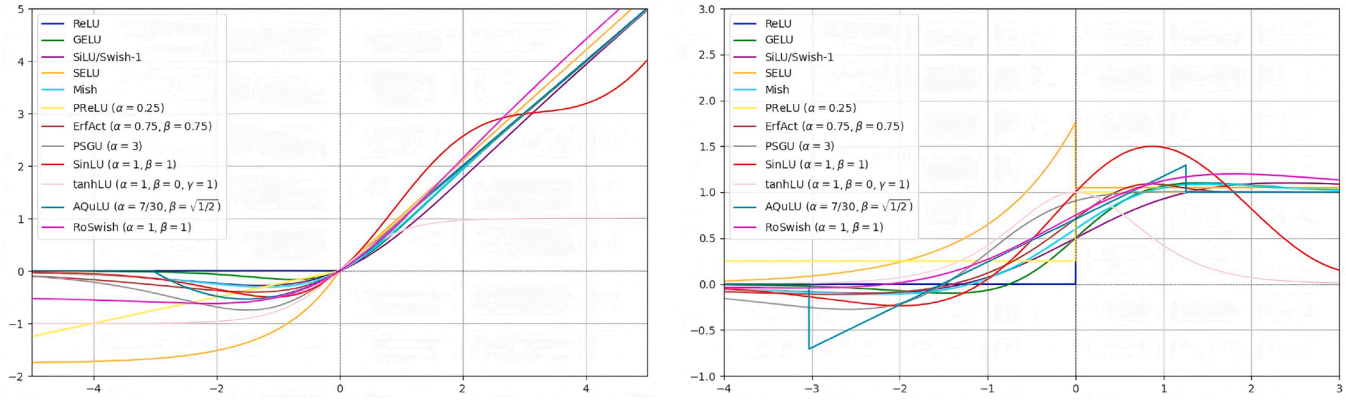


Fig. 3. The original functions (Left) and derivatives (Right).

Elements of the parameter space θ are typically optimized independently during training, thus $|\Sigma|$ can be expressed as:

$$|\Sigma| = det(\Sigma) = \sigma_{\theta_1}^2 \cdot \sigma_{\theta_2}^2 \cdot \cdots \cdot \sigma_{\theta_d}^2 \tag{27}$$

When parameter optimization is sufficient, $\epsilon \to 0$, $|\Sigma| \to 0$, and the information gain tends to maximize.

In summary, the information gain of fixed AFs is $\Delta I = 0$, because $H(Y|X,\theta) = H(Y|X)$. Adaptive AFs significantly reduce the conditional entropy $H(Y|X,\theta)$ through dynamic parameter optimization, thereby achieving a final information gain of $\Delta I \geq -\frac{1}{2}\log|\Sigma| - C \cdot \epsilon > 0$.

6. Experiment

This section presents the experimental task configurations and results used to evaluate the performance of RoSwish, as well as extensive ablation experiments targeting its properties. Since many AFs have been proposed, this article selects five fixed AFs, including ReLU (Glorot et al., 2011), GELU (Hendrycks & Gimpel, 2016), SiLU (Elfwing et al., 2018), SELU (Klambauer et al., 2017) and Mish (Misra, 2019), eight parametric AFs, PReLU (He et al., 2015), ErfAct (Biswas et al., 2022), Swish (Ramachandran et al., 2017), Parameterized Self-circulating Gating Unit (PSGU) (Li et al., 2021), Sinu-sigmoidal Linear Unit (SinLU) (Paul et al., 2022), tanhLU (Shen et al., 2022), Adaptive Offset AF (AOAF) (Jiang et al., 2022) and Adaptive Quadratic Linear Unit (AQuLU) (Wu et al., 2023) are included as benchmark functions for comparison, whose original and derivative functions are shown in Fig. 3.

6.1. Performance evaluation experiment

This paper evaluates the performance of RoSwish on the following tasks:

- MNIST classification (grayscale images with 10 classes, 60k training examples and 10k test examples)
- MNIST autoencoding
- Tweet part-of-speech tagging (1000 training, 327 validation, and 500 testing tweets)
- ETTm2 prediction (41,692 training, 13,897 validation, 13,899 test time points)

All neural networks are trained using the Adam optimizer (Kingma, 2014). All experiments are implemented in PyTorch (Paszke et al., 2019) and conducted on a NVIDIA Geforce RTX 4090 24GB GPU. To validate the performance improvements of RoSwish compared to existing AFs, this study replicates the official experimental setup of GELU (Hendrycks & Gimpel, 2016) and also considers time series prediction tasks to further explore the universality of RoSwish. The difference from the GELU experiment lies in the weight initialization, which adopts the default Kaiming initialization method for linear layers (He et al., 2015). The learning rates (LRs) are adjusted to $\{1e-3, 1e-4, 1e-5\}$ to evaluate the gradient stability of the RoSwish network. In the MNIST Classification and ETTm2 Prediction tasks, the effect of RoSwish on dropout is demonstrated based on a learning rate of 1e-3 to assess its self-regularization capability. The median results of five runs are taken for each experiment, with the best results highlighted in bold and the second-best results underlined. The lower Log Loss, Error, MSE or MAE indicates better AFs performance. "-" indicates that the AF-based neural network does not converge. Note that experiments without explicitly stated dropout values default to 0.

6.1.1. MNIST classification

The image classification experiments utilize a fully connected neural network with 8 hidden layers, each containing 128 neurons. The model is trained for 50 epochs with a batch size of 128, and the loss function used is Log Loss.

6.1.2. MNIST autoencoder

The image autoencoding experiments employ a network with neurons sequentially structured as 1000, 500, 250, 30, 250, 500, and 1000. The model is trained for 70 epochs with a batch size of 64, and the loss function used is MSE.

6.1.3. Twitter Pos tagging

The natural language processing experiments employ a tweet annotator with merely two layers, each consisting of 256 neurons (Gimpel et al., 2011; Owoputi et al., 2013). The model is trained for 30 epochs with a batch size of 16 and a dropout rate of 0.8, and the loss function used is Log Loss.

6.1.4. ETTm2 prediction

The time series forecasting experiments employ a fully connected neural network comprising 8 hidden layers, each with 64 neurons. The model is trained for 50 epochs with a batch size of 32, where both the input length and the forecast length are set to 96, and the loss function used is MSE.

6.1.5. Performance analysis

As can be seen from Tables 1,2,3,4,5,6, in all the experiments considered, regardless of whether the learning rate is set too high or too low, RoSwish effectively adapts and achieves stable performance improvements. This demonstrates that RoSwish outperforms other AFs in maintaining neural network stability. As shown in Tables 1–2 and 5–6, whether dropout is introduced or not, RoSwish matches or surpasses other AFs, indicating its superior self-regularization capability. Based on the specific results, it can be analyzed that under the benchmarks of learning rates 1e-3, 1e-4, and 1e-5, RoSwish provides reductions of

Table 1
MNIST classification test set results.

LRs	1e-3		1e-4		1e-5	
AFs	Log Loss	Error	Log Loss	Error	Log Loss	Error
ReLU	0.1172	0.0225	0.1250	0.0293	0.2616	0.0798
GELU	0.1100	0.0210	0.1184	0.0254	0.2455	0.0722
SiLU	0.1024	0.0221	0.1002	0.0244	0.2635	0.0796
SELU	0.1110	0.0193	0.1174	0.0262	0.1538	0.0464
Mish	0.1217	0.0207	0.1079	0.0257	0.2564	0.0768
PReLU	0.1078	0.0210	0.1275	0.0281	0.3150	0.0932
ErfAct	–	–	0.1161	0.0248	0.2116	0.0641
Swish	0.1100	0.0206	0.1096	0.0260	0.2678	0.0813
PSGU	0.1079	0.0206	0.1228	0.0263	0.2090	0.0634
AOAF	0.1214	0.0221	0.1502	0.0346	0.3359	0.0926
SinLU	0.1108	0.0223	0.1053	0.0224	**0.1233**	0.0373
tanhLU	0.1142	0.0248	0.1152	0.0267	0.1744	0.0478
AQuLU	0.1001	0.0204	0.1203	0.0263	0.2298	0.0673
RoSwish	**0.0957**	**0.0185**	**0.0999**	**0.0212**	0.1237	**0.0365**

Table 2
MNIST classification test set results with dropout of 0.5.

AFs	Log Loss	Error
ReLU	1.2295	0.5835
GELU	0.5538	0.1503
SiLU	0.3094	0.0838
SELU	0.1513	**0.0343**
Mish	0.2771	0.0729
PReLU	0.1771	0.0443
ErfAct	–	–
Swish	0.2026	0.0509
PSGU	0.2030	0.0513
AOAF	0.1700	0.0359
SinLU	0.3001	0.0768
tanhLU	**0.1467**	0.0351
AQuLU	0.2704	0.0697
RoSwish	0.2738	0.0723

Table 3
MNIST autoencoder test set results.

LRs	1e-3	1e-4	1e-5
AFs	MSE	MSE	MSE
ReLU	0.0112	0.0067	0.0119
GELU	0.0051	0.0055	0.0130
SiLU	0.0052	0.0064	0.0145
SELU	0.0156	0.0073	0.0116
Mish	0.0054	0.0051	0.0111
PReLU	0.0047	0.0052	0.0123
ErfAct	**0.0041**	**0.0049**	0.0101
Swish	0.0044	0.0056	0.0134
PSGU	0.0043	0.0057	0.0101
AOAF	0.0085	0.0113	0.0138
SinLU	0.0044	0.0052	0.0100
tanhLU	0.0047	0.0054	0.0111
AQuLU	0.0042	0.0050	0.0103
RoSwish	0.0043	0.0050	**0.0095**

8.54 % (0.1001 → 0.0957, 0.0193 → 0.0185), 5.66 % (0.1002 → 0.0999, 0.0224 → 0.0212), and 1.82 % (0.1233 → 0.1237, 0.0373 → 0.0365) in MNIST Classification; -4.88 % (0.0041 → 0.0043), -2.04 % (0.0049 → 0.0050), and 5 % (0.0100 → 0.0095) in MNIST Autoencoder; 0.59 % (0.4800 → 0.4758, 0.1380 → 0.1384), 3.06 % (0.5308 → 0.5247, 0.1569 → 0.1539), and -9.05 % (1.1382 → 1.1952, 0.3115 → 0.3241) in Twitter Pos Tagging; -13.42 % (0.0609 → 0.0666, 0.1725 → 0.1795), 3.45 % (0.1233 → 0.1200, 0.2320 → 0.2302), and 2.61 % (0.1478 → 0.1450, 0.2506 → 0.2488) in ETTm2 Prediction.

Table 4
Twitter Pos tagging test set results.

LRs	1e-3		1e-4		1e-5	
AFs	Log Loss	Error	Log Loss	Error	Log Loss	Error
ReLU	0.5172	0.1398	0.5730	0.1710	1.7676	0.4666
GELU	0.4800	0.1405	0.5774	0.1734	1.7068	0.4744
SiLU	0.4852	0.1408	0.5893	0.1774	1.7718	0.5147
SELU	0.5378	0.1518	0.5382	0.1574	**1.1382**	**0.3115**
Mish	0.4847	0.1426	0.5725	0.1676	1.6024	0.4267
PReLU	0.4973	0.1388	0.5636	0.1630	1.4883	0.3949
ErfAct	0.4834	0.1386	0.5575	0.1644	1.4383	0.3814
Swish	0.4810	0.1404	0.5900	0.1787	1.7295	0.4824
PSGU	0.4878	0.1443	0.5580	0.1633	1.3184	0.3528
AOAF	0.4840	**0.1380**	0.5715	0.1686	1.8439	0.4855
SinLU	0.4833	0.1435	0.5308	0.1569	1.2106	0.3307
tanhLU	0.4947	0.1449	0.5413	0.1591	1.1598	0.3220
AQuLU	0.4812	0.1409	0.5590	0.1647	1.3982	0.3688
RoSwish	**0.4758**	0.1384	**0.5247**	**0.1539**	1.1952	0.3241

Table 5
ETTm2 prediction test set results.

LRs	1e-3		1e-4		1e-5	
AFs	MSE	MAE	MSE	MAE	MSE	MAE
ReLU	0.0884	0.2020	0.1367	0.2431	0.1728	0.2733
GELU	0.0704	0.1822	0.1354	0.2418	0.1739	0.2729
SiLU	0.0790	0.1933	0.1388	0.2438	0.1786	0.2828
SELU	0.0868	0.2015	0.1351	0.2399	0.1495	0.2506
Mish	0.0773	0.1909	0.1378	0.2424	0.1671	0.2701
PReLU	0.0767	0.1901	0.1308	0.2395	0.1620	0.2626
ErfAct	0.0688	0.1824	0.1365	0.2427	0.1568	0.2576
Swish	0.0772	0.1913	0.1402	0.2460	0.1781	0.2823
PSGU	0.0729	0.1877	0.1375	0.2429	0.1593	0.2600
AOAF	0.0940	0.2074	0.1428	0.2520	0.1801	0.2842
SinLU	**0.0609**	**0.1725**	0.1233	0.2320	0.1478	0.2511
tanhLU	0.0620	0.1728	0.1261	0.2328	0.1565	0.2564
AQuLU	0.0798	0.1934	0.1406	0.2429	0.1570	0.2580
RoSwish	0.0666	0.1795	**0.1200**	**0.2302**	**0.1450**	**0.2488**

Table 6
ETTm2 prediction test set results with dropout of 0.5.

AFs	MSE	MAE
ReLU	0.0824	0.1931
GELU	0.0810	0.1947
SiLU	0.0853	0.1999
SELU	0.0892	0.2034
Mish	0.0798	0.1945
PReLU	0.0870	0.1962
ErfAct	0.0794	0.1929
Swish	0.0856	0.1990
PSGU	0.0821	0.1971
AOAF	0.0877	0.2000
SinLU	**0.0711**	**0.1856**
tanhLU	0.0748	0.1895
AQuLU	0.0916	0.2065
RoSwish	0.0763	0.1894

6.2. Ablation experiment

6.2.1. Accuracy evaluation

In order to verify the accuracy advantage of RoSwish, the parallel computing operation of GPU is not considered. All experiments are conducted on an AMD Ryzen 5 5600H CPU. All datasets and task configurations remain unchanged.

As shown in Table 7, RoSwish demonstrates performance improvements across all datasets and tasks compared to other AFs. This indicates that the proposed RoSwish significantly outperforms existing AFs in terms of accuracy, further highlighting the effectiveness and advancement of its integration of multiple features.

Table 7
All task test set results.

Experiments	MNIST Classification		MNIST Autoencoder	Twitter Pos Tagging		ETTm2 Prediction	
LRs	1e-3		1e-4	1e-3		1e-3	
AFs	Log Loss	Error	MSE	Log Loss	Error	MSE	MAE
ReLU	0.1240	0.0229	0.0078	0.5089	0.1379	0.0875	0.2019
GELU	0.1094	0.0211	0.0057	0.4827	0.1395	0.0729	0.1862
SiLU	0.1174	0.0217	0.0055	0.4823	0.1440	0.0780	0.1922
SELU	0.1067	0.0193	0.0073	0.5369	0.1506	0.0890	0.2034
Mish	0.1169	0.0225	0.0051	0.4838	0.1450	0.0728	0.1874
PReLU	0.1107	0.0221	0.0052	0.4923	0.1398	0.0785	0.1910
ErfAct	–	–	**0.0049**	0.4825	0.1411	0.0688	0.1828
Swish	0.1101	0.0209	0.0062	0.4752	0.1393	0.0772	0.1908
PSGU	0.1131	0.0222	0.0056	0.4843	0.1447	0.0760	0.1883
AOAF	0.1078	0.0205	0.0082	0.4881	0.1372	0.0944	0.2062
SinLU	0.1182	0.0238	0.0052	0.4812	0.1426	**0.0621**	**0.1733**
tanhLU	0.1083	0.0249	0.0054	0.4976	0.1456	0.0631	0.1741
AQuLU	**0.1044**	0.0196	0.0050	0.4795	0.1419	0.0785	0.1934
RoSwish	0.1064	**0.0192**	0.0050	**0.4747**	**0.1352**	0.0678	0.1807

Table 8
MNIST classification test set results on deep RoSwish network.

Hidden Layers	18		26		32	
AFs	Log Loss	Error	Log Loss	Error	Log Loss	Error
ReLU	0.1479	0.0377	–	–	–	–
GELU	0.1354	0.0333	–	–	–	–
SiLU	0.1764	0.0445	–	–	–	–
SELU	0.1082	0.0282	0.1111	0.0278	**0.1141**	**0.0293**
Mish	0.1733	0.0421	–	–	–	–
PReLU	0.1286	0.0317	–	–	–	–
ErfAct	–	–	–	–	–	–
Swish	0.1677	0.0412	–	–	–	–
PSGU	0.1570	0.0372	0.1777	0.0456	–	–
AOAF	0.1537	0.0358	–	–	–	–
SinLU	0.1083	0.0233	0.1251	**0.0241**	–	–
tanhLU	0.1278	0.0306	0.3187	0.0683	–	–
AQuLU	0.1578	0.0399	–	–	–	–
RoSwish	**0.1016**	**0.0221**	**0.1090**	0.0260	0.1481	0.0340

Table 9
CIFAR-10 classification test set results.

AFs	Log Loss	Error
ReLU	0.4400	0.0848
GELU	0.4423	0.0837
SiLU	0.4601	0.0878
SELU	0.4310	0.0867
Mish	0.4445	0.0873
PReLU	0.4418	0.0850
ErfAct	–	–
Swish	0.4507	0.0843
PSGU	0.4354	0.0855
AOAF	–	–
SinLU	0.4350	**0.0833**
tanhLU	0.5104	0.1030
AQuLU	0.4452	0.0862
RoSwish	**0.4307**	0.0841

Table 10
Mutual information for all AFs.

AFs	MI (nat)
ReLU	0.5165
GELU	0.5360
SiLU	0.5375
SELU	0.5384
Mish	0.5382
PReLU	0.5534
ErfAct	0.5464
Swish	0.5389
PSGU	0.5484
AOAF	0.5465
SinLU	0.5505
tanhLU	0.5621
AQuLU	0.5278
RoSwish	**0.5657**

6.2.2. *Deep network stability*

The MNIST Classification experiment employs three fully connected neural networks with 18, 26, and 32 hidden layers respectively to validate the stability of the RoSwish network. The learning rate is set to 1e-4, while other configurations remain unchanged.

As shown in Table 8, in the neural network with 18 hidden layers, RoSwish outperforms other AFs. However, as the number of neural network layers continues to increase, and when the number of hidden layers reaches 26, only SELU, PSGU, SinLU, tanhLU, and RoSwish are able to achieve convergence. This indicates that RoSwish has higher stability in constructing deep neural networks. Notably, in the neural network with 32 hidden layers, PSGU, SinLU, and tanhLU no longer converge, and the accuracy of SELU, based on self-normalizing properties, surpasses that of RoSwish. This finding further confirms the robustness of self-normalizing properties in deep neural network architectures. Therefore, exploring and realizing a RoSwish with self-normalizing capabilities is a new subject worthy of in-depth research.

6.2.3. *Noise robustness*

The CIFAR-10 classification experiment replicates the 9-layer shallow convolutional neural network of GELU (Hendrycks & Gimpel, 2016). The input data undergoes Zero-phase Component Analysis (ZCA) whitening and the addition of Gaussian noise to validate the noise robustness of RoSwish. The model is trained for 200 epochs with a batch size of 128, an initial learning rate set to 1e-3, and the loss function used is Log Loss.

As shown in Table 9, both SinLU and RoSwish significantly outperform GELU, which is based on a Gaussian distribution, further validating their superior robustness against Gaussian noise interference.

6.2.4. *Information gain*

Higher information gain indicates that AFs effectively capture key information in the input features, significantly reducing uncertainty about the target variable, thereby enhancing the model's reasoning ability. In the ETTm2 prediction task, the information gain capability of RoSwish is analyzed from the perspective of mutual information (MI), using a learning rate of 1e-3. When optimization is sufficient, the information gain reaches the theoretical maximum lower bound. Therefore, during the validation process, the MI value between the predicted output and the true target is calculated.

As shown in Table 10, the MI of adaptive AFs generally outperforms that of fixed AFs, but the performance difference is not absolute and

Table 11
Test set results for all tasks. "/" indicates the removal of parameter α or β, which means that $\alpha = 0$ or $\beta = 1$ is fixed.

Experiments		MNIST Classification		MNIST Autoencoder	Twitter Pos Tagging		ETTm2 Prediction	
LRs		1e-3		1e-4	1e-3		1e-3	
Initial Values		Log Loss	Error	MSE	Log Loss	Error	MSE	MAE
$\alpha = 0.817$	$\beta = 3.0$	**0.0957**	**0.0185**	0.0050	0.4758	0.1384	**0.0666**	**0.1795**
$\alpha = 0.817$	$\beta = 3.0$ ***(fixed)***	0.1142	0.0213	0.0052	0.5018	0.1490	0.0722	0.1852
	$\beta = 1.0$	0.1047	0.0200	0.0054	0.4828	0.1436	0.0714	0.1863
	$\beta = 2.0$	0.0992	0.0197	0.0052	0.4782	0.1405	0.0667	0.1804
	$\beta = 2.5$	0.1085	0.0208	0.0051	0.4751	0.1388	0.0670	0.1798
$\alpha = 0.817$	$\beta = /$	0.1126	0.0188	0.0055	0.4846	0.1443	0.0753	0.1893
	$\beta = 3.5$	0.1022	0.0198	0.0049	**0.4745**	0.1375	0.0707	0.1821
	$\beta = 4.0$	0.1185	0.0211	**0.0048**	0.4790	0.1379	0.0679	0.1820
	$\beta = 5.0$	0.1035	0.0204	**0.0048**	0.4779	**0.1348**	0.0682	0.1813
	$\alpha = 0.8150$	0.1078	0.0210	0.0050	0.4865	0.1399	0.0692	0.1828
	$\alpha = 0.8160$	0.1028	0.0226	0.0050	0.4774	0.1379	0.0702	0.1857
	$\alpha = 0.8165$	0.1168	0.0237	0.0051	0.4756	0.1383	0.0685	0.1811
$\beta = 3.0$	$\alpha = /$	0.1009	0.0203	0.0058	0.5023	0.1468	0.0778	0.1909
	$\alpha = 0.8175$	0.0976	0.0190	0.0050	0.4791	0.1384	0.0687	0.1813
	$\alpha = 0.8180$	0.1050	0.0200	0.0050	0.4754	0.1369	0.0667	0.1800
	$\alpha = 0.8190$	0.1148	0.0225	0.0050	0.4752	0.1362	0.0670	0.1805

Table 12
Test set results for normalized RoSwish.

Experiments	Twitter Pos Tagging		MNIST Classification	
AFs	Log Loss	Error	Log Loss	Error
SELU	1.1382	0.3115	0.1141	0.0293
RoSwish	1.1952	0.3241	0.1481	0.0340
BN + RoSwish	**0.9375**	**0.2724**	**0.0990**	**0.0210**

requires further analysis in conjunction with the specific characteristics of each AF. Among them, RoSwish achieves the optimal MI value, demonstrating its advantage in feature information extraction.

6.2.5. *Sensitivity of the RoSwish parameters*

Different parameter combinations are used to assess the model's sensitivity regarding its initial values. All datasets and task configurations remain unchanged.

As shown in Table 11, different settings of parameters α and β have a significant impact on model performance, with the combination set in this paper performing best across multiple metrics. In the MNIST Autoencoder task, increasing the initial value of β helps to further reduce errors. In the Twitter Pos Tagging task, appropriately increasing the values of α and β can enhance model performance. Additionally, fixing parameters leads to a decline in model performance, further emphasizing the crucial role of parameter flexibility in the optimization process.

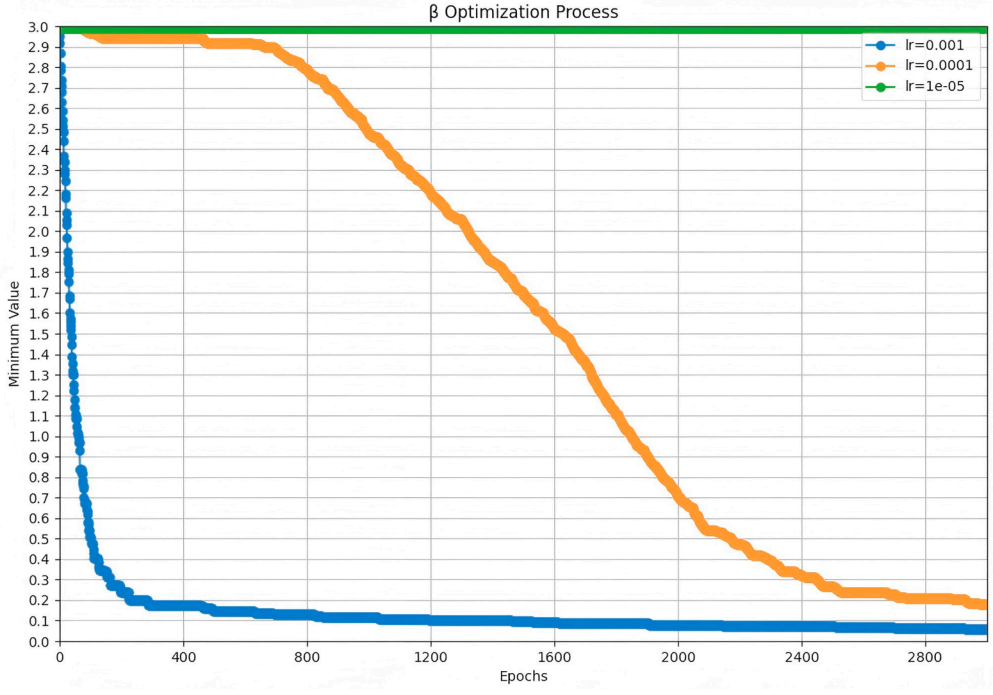


Fig. 4. The minimum value curve of β during the training process.

6.2.6. Normalized RoSwish

Some experiments have shown that self-normalizing SELU outperforms RoSwish in both Twitter Pos Tagging (with a learning rate of 1e-5 in Table 4) and MNIST Classification (with 32 hidden layers in Table 8). Therefore, this paper investigates whether RoSwish based on batch normalization (BN) can further enhance network performance.

From Table 12, it can be seen that normalized RoSwish can significantly improve network performance. By introducing batch normalization, RoSwish gives a 30.19 % reduction (1.1382 → 0.9375, 0.3115 → 0.2724) in Twitter Pos Tagging, and 41.56 % (0.1141 → 0.0990, 0.0293 → 0.0210) in MNIST Classification.

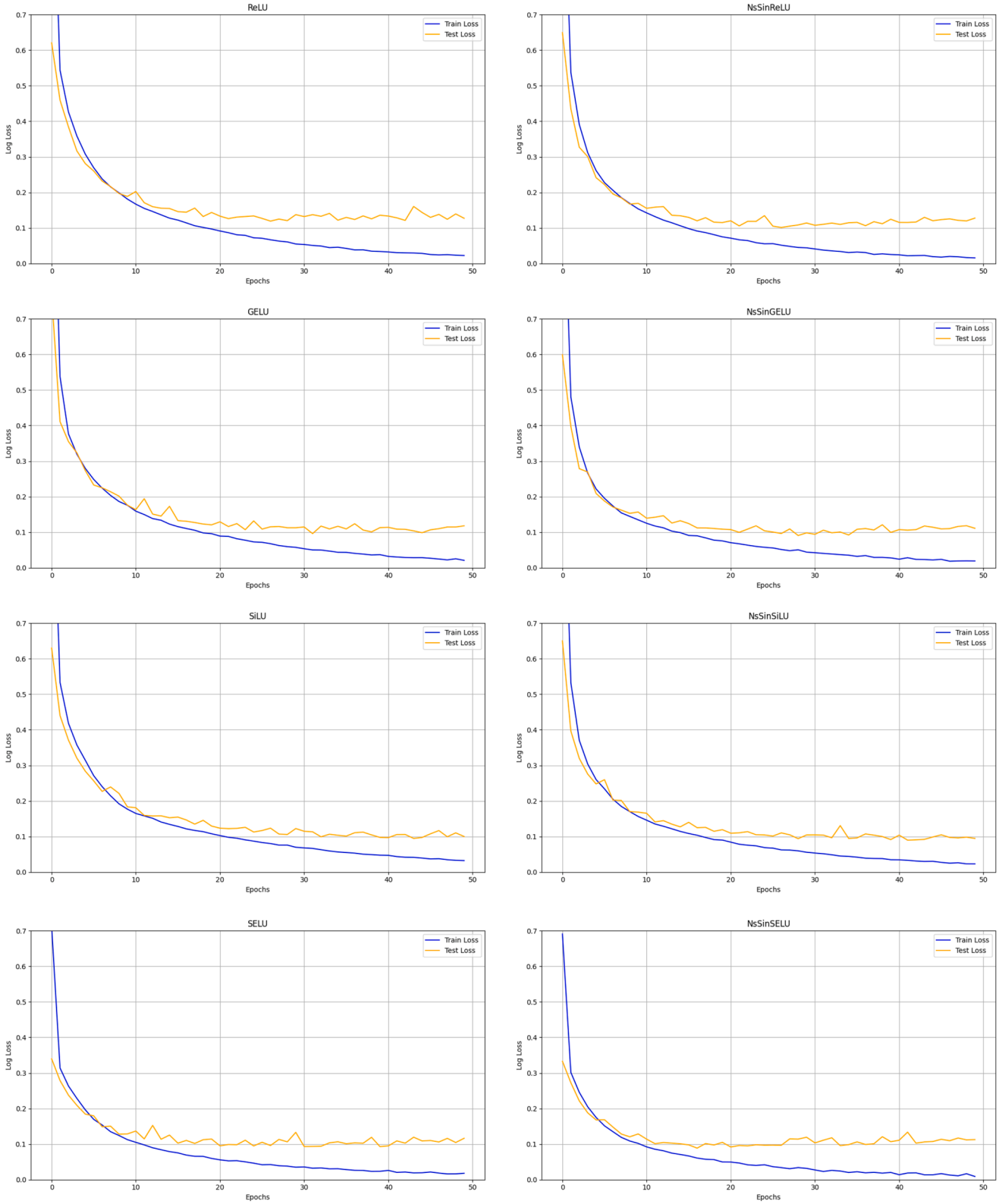


Fig. 5. Log loss curves for all AFs.

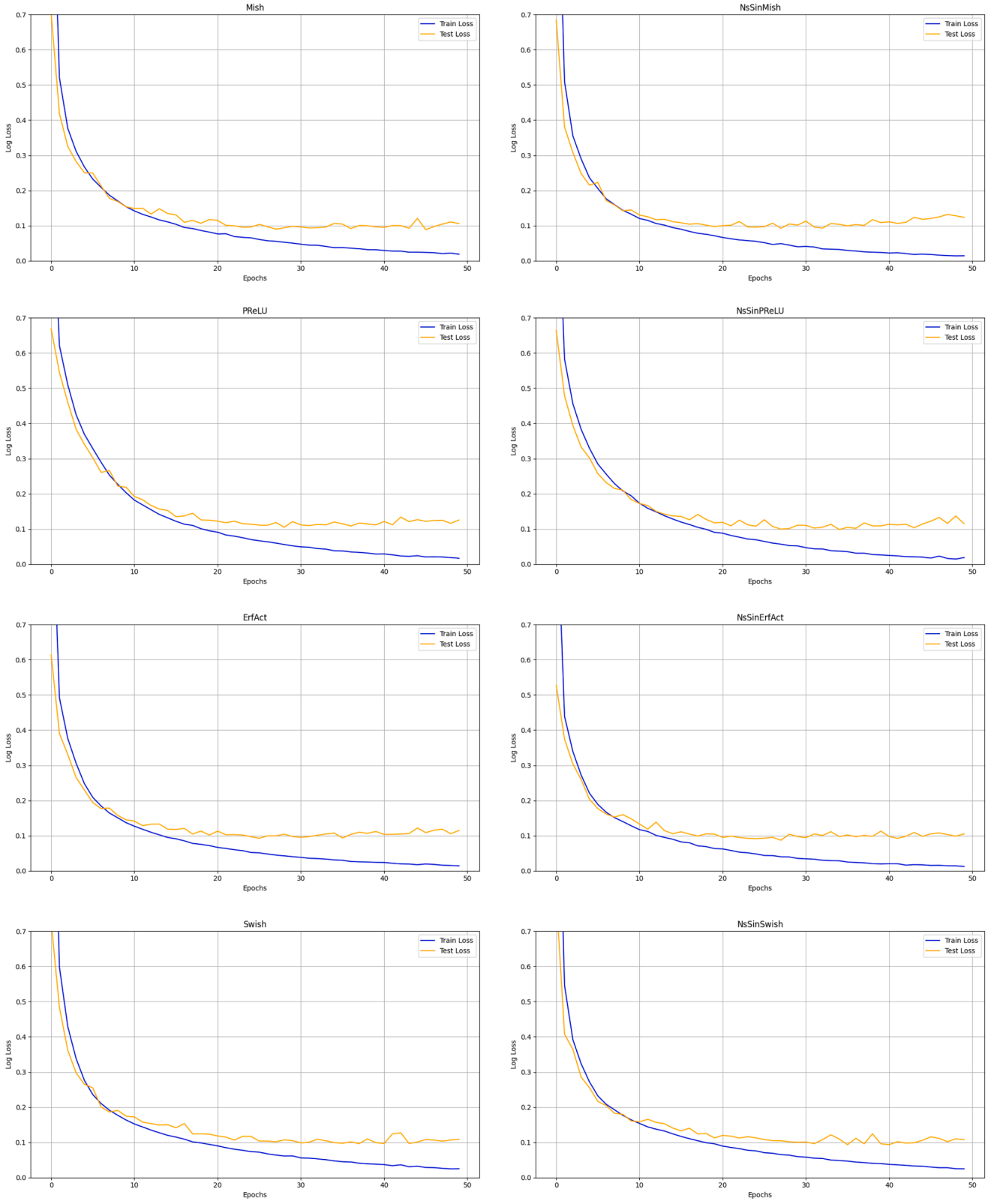


Fig. 5. Continued.

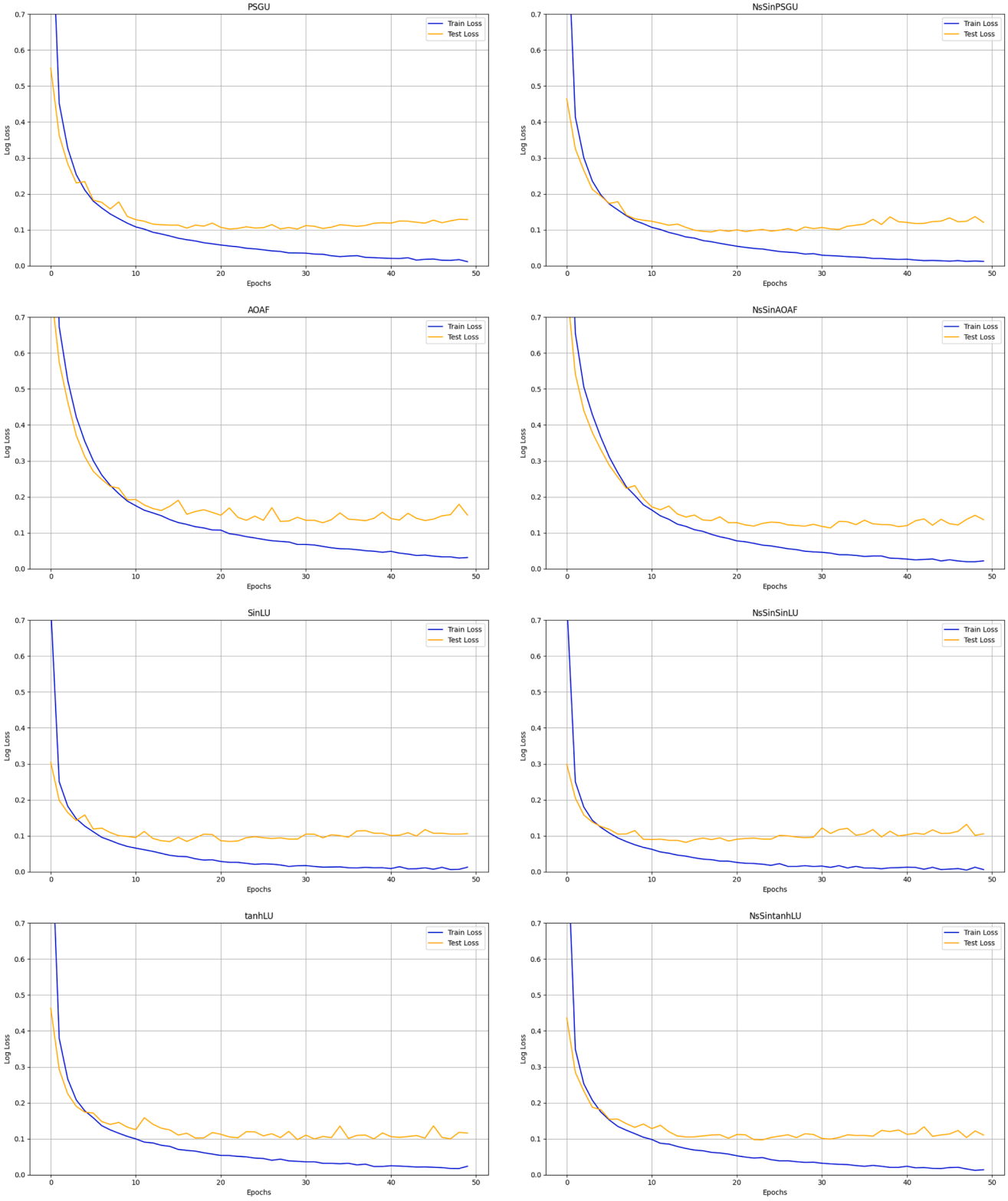


Fig. 5. Continued.

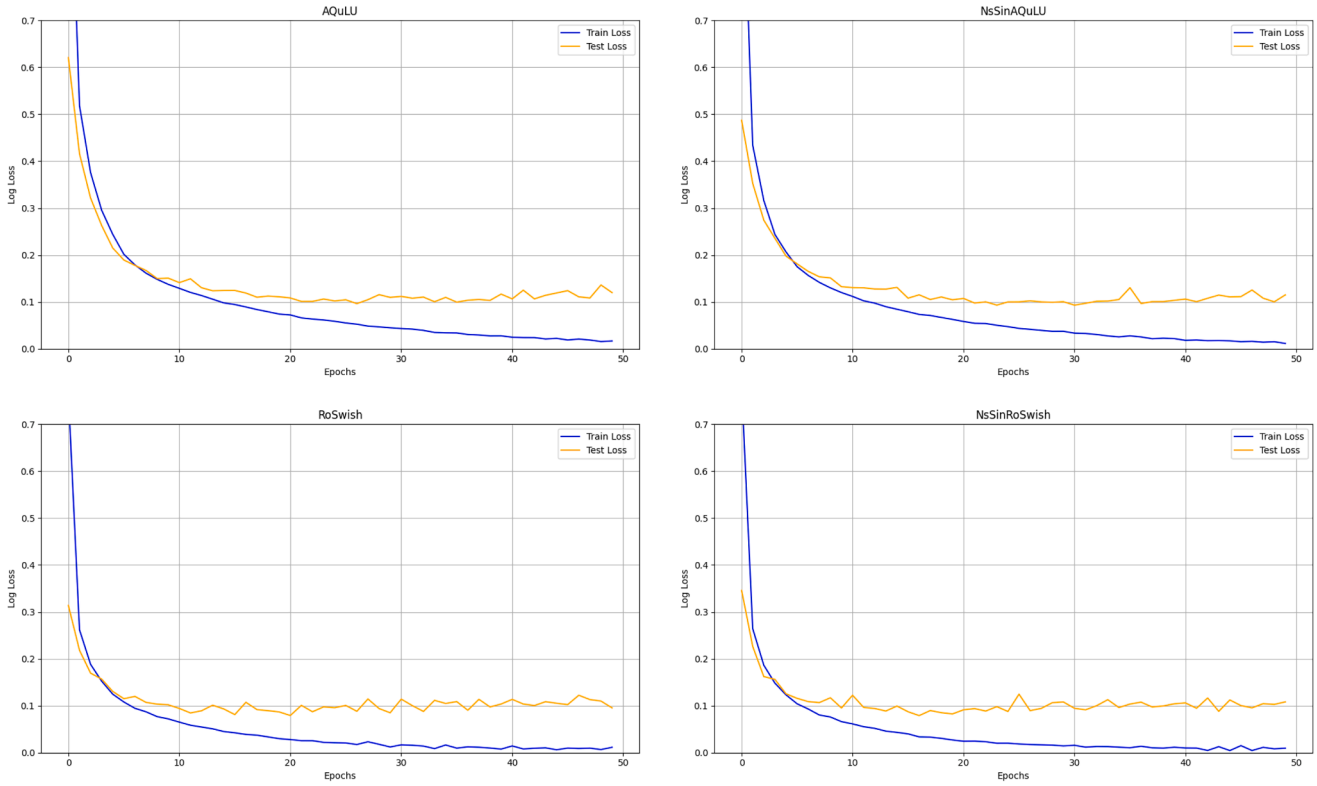


Fig. 5. Continued.

6.2.7. Experimental verification of Lemma 1

To validate the conjecture in Lemma 1, experiments were conducted in Twitter Pos Tagging. It is important to note that as the number of training iterations increases, the parameter space β generally shows a decreasing trend. If, after extensive training, the minimum value of this space can converge to a definite positive number, this will provide strong evidence for the conjecture in Lemma 1, thereby ensuring the theoretical rigor of the universal function approximation capability of the RoSwish network. Given that the learning rate has a direct impact on the parameter optimization results, three representative learning rates ($lrs \in \{1e-3, 1e-4, 1e-5\}$) were adopted to comprehensively analyze the parameter optimization process. This experiment extended the training epochs from 30 to 3000 to simulate the "at infinity" condition. To facilitate monitoring and analysis, the β value was recorded after each parameter update, and the minimum β value within each training epoch was used as the observation metric.

As shown in Fig. 4, the higher the learning rate, the faster the convergence speed of the minimum value of the parameter space β, and the smaller the final convergence value. When the learning rate is set to 1e-3, the minimum value of β stably converges to a strictly positive number, consistent with the conjecture of Lemma 1. It is noteworthy that modern complex models typically adopt lower learning rates, at which the convergence value of β is relatively larger, further enhancing the rigor of the conjecture. For special scenarios with high learning rate, the performance of the model can be maintained by moderately increasing the initial value of β (See Table 11) to balance theoretical rigor and application efficiency.

6.2.8. Generalization capability

Considering that a model with strong generalization capability can more efficiently adapt to new data, thereby exhibiting higher stability and reliability in practical applications. Therefore, in the MNIST Classification task (taking a learning rate of 1e-4 as an example), the training and test log loss curves for all AFs are plotted, with each curve representing the median result of 5 runs.

As shown in Fig. 5, among the existing original functions, the RoSwish network exhibits the fastest convergence speed, with both its training and test errors reaching the lowest levels, demonstrating the superior learning characteristics of the proposed AFs. It is worth noting that when NsSinAFs introduce the sine waves into the original AFs, although the error decline trajectory of the network in the training stage is basically consistent with the original function, it shows stronger generalization ability on the test set. This indicates that sine waves help enhance the non-stationary information captured by the activation function, effectively improving the model's ability to express new data.

6.2.9. Efficiency analysis

Considering the stringent requirements for network training efficiency in real-time application scenarios and resource-constrained environments, a comprehensive comparison of training time and memory usage for all AFs was conducted in the MNIST classification task. For training time, the average execution time per epoch was calculated, and the median result of five experiments was taken. In terms of memory, memory allocation during the training phase of the experiment was recorded.

As shown in Fig. 6, RoSwish demonstrates good performance in both training time and memory usage, while NsSinAFs shows some increase in training time but maintains stable memory usage. If computational time is relatively sufficient, the accuracy advantages of NsSinAFs in strongly non-stationary data and complex models can be considered.

7. Result and discussion

All experiments validate that RoSwish's performance is superior to existing AFs, which is precisely due to its ability to approximate and fit them, showcasing their respective characteristics and there by enhancing the neural network's capability to approximate nonlinear information. As a trainable AF, RoSwish can dynamically adjust its activation

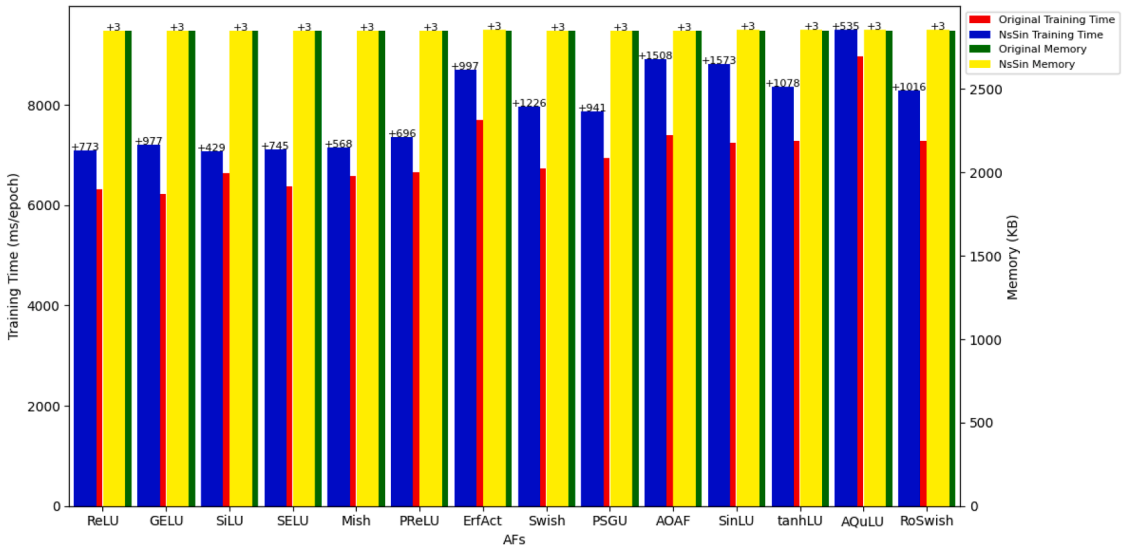


Fig. 6. Training time and memory of all AFs.

patterns according to different datasets and tasks, equivalent to optimizing the optimal combination of fixed AFs.

Although there are already various types of AFs with outstanding features and advantages, the experimental results in this paper indicate that comprehensive AFs can further improve neural network performance. The only challenge is that RoSwish cannot achieve self-normalizing neural networks like SELU, as its mapping rules continuously adapt during the training process.

When training RoSwish, it is recommended to prioritize momentum optimization methods, such as the standard Adam optimizer (Kingma, 2014), over traditional gradient descent methods. Research shows that the inputs to neurons often approximate a Gaussian distribution, and the Sigmoid function can be considered an approximate form of the cumulative distribution function of a Gaussian distribution (Hendrycks & Gimpel, 2016). Therefore, in future research aimed at enhancing neural network performance, choosing RoSwish based on the cumulative distribution function of a Gaussian distribution may be an effective direction, albeit at the cost of significantly increasing the computational complexity of relatively simple neural networks. The experiments in this paper did not use $x \cdot sigmoid(\beta \cdot x) + 0.5 \cdot \alpha \cdot \tanh(0.5 \cdot \beta \cdot x)$ because it is equivalent to $(x + \alpha) \cdot sigmoid(\beta \cdot x) - 0.5 \cdot \alpha$, but the latter's mathematical expression is simpler to compute.

8. Conclusion and future work

In this study, a comprehensive RoSwish activation function is proposed, and its design concept integrates the characteristics and advantages of multiple activation functions. Although the current version of RoSwish does not yet have the self-normalization, in fields such as computer vision, natural language processing, and time series prediction, RoSwish has significantly improved the performance of models, fully demonstrating its applicability in multi-task scenarios and the robustness of the models.

Aiming at the inherent limitations of neural networks when processing non-stationary signals, this study has designed a series of activation functions based on non-stationary sine waves. The results of the MNIST classification experiment show that this new type of activation function can not only effectively capture the non-stationary fluctuation characteristics of signals but also significantly enhance the extrapolation ability of neural networks.

Looking ahead to follow-up research, the focus will be on the following key directions: Firstly, efforts will be made to enable the RoSwish network to naturally converge to the theoretical stable state, thus achieving self-normalization. Secondly, given the powerful generalization ability of non-stationary sine activation functions, a systematic evaluation of their modeling effectiveness will be carried out, and solutions for deeply integrating them with mainstream network architectures will be explored. Finally, attempts will be made to explore the pre-trained version of the adaptive activation function, that is, through pre-training on a large-scale pre-training dataset to obtain optimized initial parameters, thereby effectively improving the training efficiency and generalization ability. It is expected that this research idea can fill the technical gap between fixed activation functions and fully trainable activation functions, and open up a new path for the efficient optimization of deep learning.

CRediT authorship contribution statement

Shengbo Zhang: Writing – original draft, Methodology, Writing – review & editing, Visualization, Data curation, Software, Conceptualization; **Guoqiang Ren:** Writing – review & editing, Resources, Formal analysis, Validation, Project administration, Supervision.

Data availability

Public data, download link has been provided in the paper.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Series of non-stationary sine AFs

A.1. Proposed non-stationary sine AFs

In the inductive reasoning process of neural networks, although unknown signals share statistical similarities with training data, their time-varying characteristics often accompany the emergence of unobserved patterns. It is noteworthy that the statistical properties of actual signals typically exhibit non-stationarity, with their spectral energy showing significant differences between high and low frequency domains. To address this, this paper introduces non-stationary sine waves to the input of AFs, designing a series of AFs based on non-stationary sine waves (called

Table A.1
MNIST classification test set results.

Versions	Original		NsSin	
AFs	Log Loss	Error	Log Loss	Error
ReLU	**0.1250**	0.0293	0.1287	**0.0277**
GELU	0.1184	0.0254	**0.1110**	**0.0251**
SiLU	0.1002	0.0244	**0.0947**	**0.0234**
SELU	0.1174	0.0262	**0.1127**	**0.0240**
Mish	**0.1079**	**0.0257**	0.1237	0.0262
PReLU	0.1275	0.0281	**0.1148**	**0.0254**
ErfAct	0.1161	0.0248	**0.1075**	**0.0233**
Swish	0.1096	**0.0260**	**0.1077**	0.0278
PSGU	0.1228	0.0263	**0.1206**	**0.0216**
AOAF	0.1502	0.0346	**0.1364**	**0.0299**
SinLU	0.1053	0.0224	**0.1046**	**0.0204**
tanhLU	0.1152	0.0267	**0.1103**	**0.0238**
AQuLU	0.1203	0.0263	**0.1151**	**0.0243**
RoSwish	**0.0957**	**0.0185**	0.1050	0.0201

Table A.2
Exchange rate prediction test set results.

Versions	Original		NsSin	
AFs	MSE	MAE	MSE	MAE
ReLU	**0.0483**	0.1386	0.0521	**0.1308**
GELU	0.0334	0.1126	**0.0305**	**0.1028**
SiLU	0.0586	0.1450	**0.0287**	**0.1134**
SELU	0.2568	0.3578	**0.1921**	**0.3087**
Mish	0.0347	0.1091	**0.0313**	**0.1046**
PReLU	**0.0641**	**0.1588**	0.0787	0.1780
ErfAct	0.0574	0.1507	**0.0311**	**0.1071**
Swish	0.0451	0.1269	**0.0368**	**0.1217**
PSGU	**0.0366**	**0.1187**	0.0519	0.1489
AOAF	0.0480	0.1394	**0.0393**	**0.1292**
SinLU	0.0381	0.1228	**0.0350**	**0.1164**
tanhLU	0.1564	0.2461	**0.1121**	**0.2041**
AQuLU	0.0535	0.1512	**0.0428**	**0.1287**
RoSwish	**0.0350**	**0.1082**	0.0535	0.1408

$NsSinAFs(f)$), where $f = a \cdot x + b \cdot \sin(x)$, a and b represent channel-wise trainable parameters, starting from 1 and 0, respectively. When $a = 1$ and $b = 0.05$, the original and derivative functions of are as shown in Fig. A.1. The performance of NsSinAFs is evaluated in the MNIST Classification task, with the learning rate set to 1e-4 and other configurations remaining unchanged. The lower Log Loss or Error indicates better AFs performance.

As shown in Table A.1, NsSinAFs demonstrate a significant performance improvement over the original AFs, with an average enhancement of 6.55 % (a 1.08 % reduction in log loss and a 5.47 % reduction in error). This further indicates that by extracting non-stationary fluctuating signals, AFs can effectively enhance network accuracy.

A.2. Theoretical analysis of nonstationary sine signals

In the field of signal processing, a random signal is defined as non-stationary if its statistical properties (such as mean, variance) evolve over time, or if its autocorrelation function depends not only on the time difference but also on absolute time. As shown in Fig. A.2, both the mean and variance of the parameter β present significant time-varying characteristics, according to which $\beta(t)$ can be strictly judged to be a non-stationary process. The following further demonstrates the non-stationary properties of the introduced sinusoidal signal $X(t)$.

Theorem 5. *Suppose the random process $\theta(t)$ is defined in the time domain $t \in [0, +\infty)$, and the sinusoidal signal is constructed as $X(t) = \beta(t) \cdot \sin(\theta(t))$, then $X(t)$ has non-stationality regardless of whether $\theta(t)$ is stationary or non-stationary.*

Proof. Take the mean value characteristics as the starting point for analysis:

$$E[X(t)] = E[\beta(t) \cdot \sin(\theta(t))] \tag{A.1}$$

Since $\beta(t)$ is itself a non-stationary process, its statistical properties are explicitly dependent on time t. In the back-propagation gradient calculation, the gradient of $\theta(t)$ is coupled with $\beta(t)$, resulting in the probability distribution of $sin(\theta(t))$ being associated with the time-varying properties of $\beta(t)$. Therefore, the expectation $E[X(t)]$ must imply a time dependence, so that the signal mean cannot be kept constant. According to the definition of non-stationary signal, the mean non-constancy of $X(t)$ directly proves its non-stationary nature. □

To sum up, Theorem 5 shows that the introduced sinusoidal signal is always non-stationary regardless of the stationarity of the input of the activation layer. This feature introduces enhanced non-stationary information into the network.

A.3. Case study

In order to verify that enhanced non-stationary information can effectively improve the network's ability to express non-stationary features, such functions will be applied to Exchange Rate real-world data (Wu et al., 2021). This dataset records the daily exchange rates of eight different countries from 1990 to 2016. Influenced by complex economic factors, exchange rate data typically lacks obvious periodicity, and statistical characteristics such as mean and variance can change signifi-

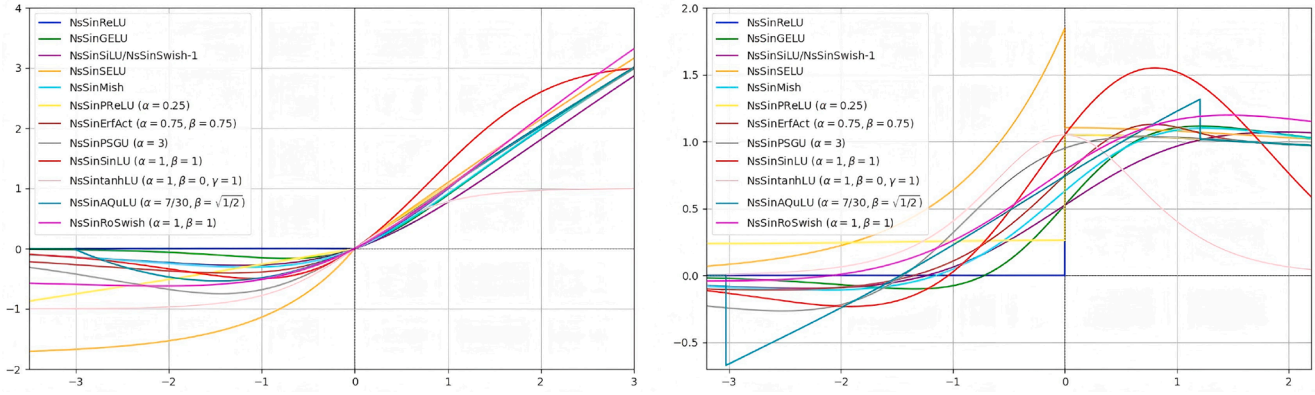


Fig. A.1. The original functions (Left) and derivatives (Right) of NsSinAFs.

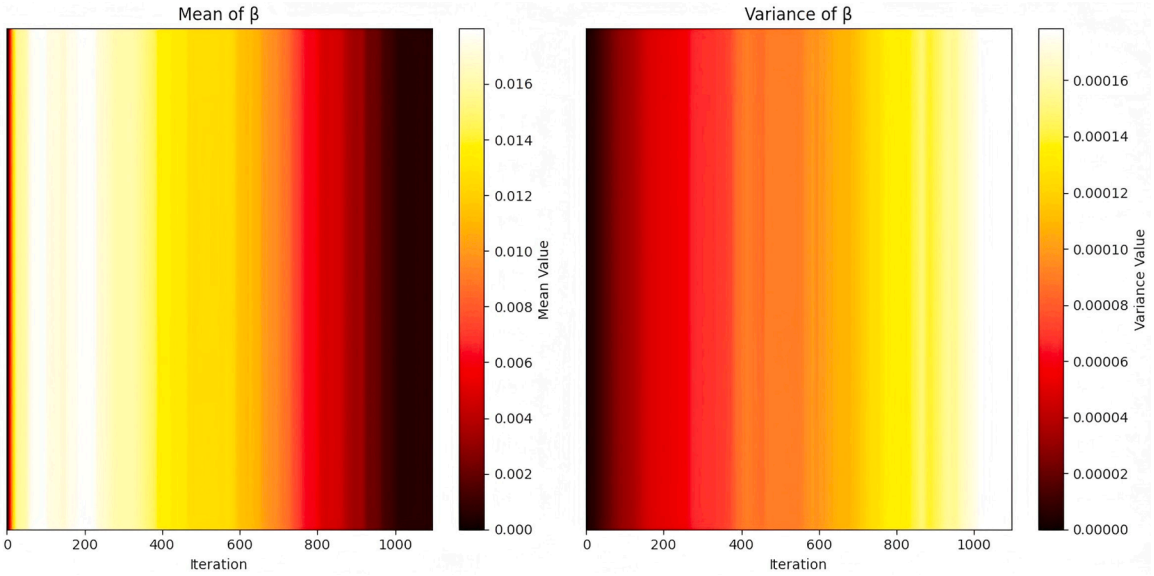


Fig. A.2. Heatmaps of the mean and variance of β in the last activation layer after 1096 training iterations (2 epochs).

cantly over time, exhibiting strong non-stationarity. This experiment selects the last two lesser-known countries, covering the final two years for these countries (calculated at 365 days per year). The Exchange Rate Prediction experiment employs a fully connected neural network with 8 hidden layers, each containing 256 neurons. It is trained for 100 epochs with a batch size of 32, both input and prediction lengths set to 96, a learning rate of 1e-3, and the MSE as the loss function. The training, validation, and test sets are divided in a ratio of 7:1:2. The lower MSE or MAE indicates better performance of the AFs. It is important to note that this problem is quite challenging due to the very limited amount of training data available.

As shown in Table A.2, NsSinAFs still exhibit significant performance improvements under highly non-stationary data conditions. This phenomenon not only validates the effectiveness of enhanced non-stationary information but also empirically demonstrates that NsSinAFs are beneficial for network models to adapt to complex non-stationary data patterns.

References

Agostinelli, F. (2014). Learning activation functions to improve deep neural networks. arXiv:1412.6830

Apicella, A., Donnarumma, F., Isgrò, F., & Prevete, R. (2021). A survey on modern trainable activation functions. *Neural Networks*, *138*, 14–32.

Augustine, M. T. (2024). A survey on universal approximation theorems. arXiv:2407.12895

Ba, J., & Frey, B. (2013). Adaptive dropout for training deep neural networks. *Advances in Neural Information Processing Systems*, *26*.

Biswas, K., Kumar, S., Banerjee, S., & Pandey, A. K. (2022). Erfact and pserf: Non-monotonic smooth trainable activation functions. *Proceedings of the AAAI conference on artificial intelligence*, *36*(6), 6097–6105. https://doi.org/10.1609/aaai.v36i6.20557

Clevert, D.-A. (2015). Fast and accurate deep network learning by exponential linear units (elus). arXiv:1511.07289

Dayan, P., & Abbott, L. F. (2005). Theoretical neuroscience: Computational and mathematical modeling of neural systems. MIT Press.

Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022). Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, *503*, 92–108.

Dugas, C., Bengio, Y., Bélisle, F., Nadeau, C., & Garcia, R. (2000). Incorporating second-order functional knowledge for better option pricing. *Advances in Neural Information Processing Systems*, *13*.

Elfwing, S., Uchibe, E., & Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, *107*, 3–11.

Fearnley, J., Goldberg, P., Hollender, A., & Savani, R. (2022). The complexity of gradient descent: CLS = PPAD ∩ PLS. *Journal of the ACM*, *70*(1), 1–74.

Gimpel, K., Schneider, N., O'connor, B., Das, D., Mills, D. P., Eisenstein, J., Heilman, M., Yogatama, D., Flanigan, J., & Smith, N. A. (2011). Part-of-speech tagging for twitter: Annotation, features, and experiments. In *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies* (pp. 42–47).

Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics* (pp. 315–323). JMLR Workshop and Conference Proceedings.

Godfrey, L. B. (2019). An evaluation of parametric activation functions for deep learning. In *2019 IEEE International conference on systems, man and cybernetics (SMC)* (pp. 3006–3011). IEEE.

Goodfellow, I., Warde-Farley, D., Mirza, M., Courville, A., & Bengio, Y. (2013). Maxout networks. In *International conference on machine learning* (pp. 1319–1327). PMLR.

Hahnloser, R. H. R., Sarpeshkar, R., Mahowald, M. A., Douglas, R. J., & Seung, H. S. (2000). Digital selection and analogue amplification coexist in a cortex-inspired silicon circuit. *Nature*, *405*(6789), 947–951.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision* (pp. 1026–1034).

Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (gelus). arXiv:1606.08415

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Hu, X., Niu, P., Wang, J., & Zhang, X. (2019). A dynamic rectified linear activation units. *IEEE Access*, *7*, 180409–180416.

Jiang, Y., Xie, J., & Zhang, D. (2022). An adaptive offset activation function for CNN image classification tasks. *Electronics*, *11*(22), 3799.

Kingma, D. P. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980

Klambauer, G., Unterthiner, T., Mayr, A., & Hochreiter, S. (2017). Self-normalizing neural networks. *Advances in Neural Information Processing Systems*, *30*.

Kolmogorov, A. N. (1957). On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. *Doklady Akademii nauk SSSR*, *114*(5), 953–995.

Lederer, J. (2021). Activation functions in artificial neural networks: A systematic overview. arXiv:2101.09957

Leshno, M., Lin, V. Y., Pinkus, A., & Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, *6*(6), 861–867.

Li, Z., Yang, X., Shen, K., Jiang, F., Jiang, J., Ren, H., & Li, Y. (2021). PSGU: Parametric self-circulation gating unit for deep neural networks. *Journal of Visual Communication and Image Representation*, *80*, 103294.

Maas, A. L., Hannun, A. Y., Ng, A. Y. et al. (2013). Rectifier nonlinearities improve neural network acoustic models. In *International conference on machine learning*. (*28*).

Mathematics, E. o. (Ed.) (2001). Taylor formula. EMS Press.

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, *5*, 115–133.

Misra, D. (2019). Mish: A self regularized non-monotonic activation function. arXiv:1908.08681

Owoputi, O., O'Connor, B., Dyer, C., Gimpel, K., Schneider, N., & Smith, N. A. (2013). Improved part-of-speech tagging for online conversational text with word clusters. In *Human language technologies: Conference of the North American chapter of the association of computational linguistics, proceedings, June 9–14, 2013, Westin Peachtree Plaza Hotel, Atlanta, Georgia, USA* (pp. 380–390). Association for Computational Linguistics.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. et al. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*.

Paul, A., Bandyopadhyay, R., Yoon, J. H., Geem, Z. W., & Sarkar, R. (2022). SinLU: Sinu-sigmoidal linear unit. *Mathematics*, *10*(3), 337.

Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for activation functions. arXiv:1710.05941

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, *65*(6), 386.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M. et al. (2015). Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, *115*, 211–252.

Shen, S.-L., Zhang, N., Zhou, A., & Yin, Z.-Y. (2022). Enhancement of neural networks with an alternative activation function tanhLU. *Expert Systems with Applications*, *199*, 117181.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, *15*(1), 1929–1958.

Taylor, B. (1715). Methodus incrementorum directa et inversa (direct and reverse methods of incrementation). London: Typis Pearsonianis prostant apud Gul. Innys ad Insignia Principis in Coemeterio Paulino.

Trottier, L., Giguere, P., & Chaib-Draa, B. (2017). Parametric exponential linear unit for deep convolutional neural networks. In *2017 16th IEEE International conference on machine learning and applications (ICMLA)* (pp. 207–214). IEEE.

Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in Neural Information Processing Systems*, *34*, 22419–22430.

Wu, Z., Yu, H., Zhang, L., & Sui, Y. (2023). The adaptive quadratic linear unit (AQuLU): adaptive non monotonic piecewise activation function. *Tehnički vjesnik*, *30*(5), 1469–1485.